Foundation of Cryptography, Lecture 11
Secure Computation, *Many* Parties[1]

Iftach Haitner

Tel Aviv University.

January 08, 2026

[1] Prepared: 2026/01/05,16:42:49.

Section 1

The Model

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, an algorithm taking the role of one of the parties in π is:

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, an algorithm taking the role of one of the parties in π is:

- Honest — acts exactly according to π.

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, an algorithm taking the role of one of the parties in π is:

- Honest — acts exactly according to π.
- Semi-honest — acts according to π, but might output **additional information**.

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, an algorithm taking the role of one of the parties in π is:

- Honest — acts exactly according to π.
- Semi-honest — acts according to π, but might output **additional information**.
- Malicious — acts arbitrarily.

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, an algorithm taking the role of one of the parties in π is:

- Honest — acts exactly according to π.
- Semi-honest — acts according to π, but might output **additional information**.
- Malicious — acts arbitrarily.

Real-model execution

For a protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ and inputs $x_1, \ldots, x_n \in \{0,1\}^*$, let $\mathsf{REAL}_\pi(x_1, \ldots, x_n)$ be the joint output of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_n(x_n))$.

Given a protocol π, an algorithm taking the role of one of the parties in π is:

- Honest — acts exactly according to π.
- Semi-honest — acts according to π, but might output **additional information**.
- Malicious — acts arbitrarily.

Dishonest parties (semi-honest or malicious) can cooperate

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

An oracle-aided algorithm is:

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

An oracle-aided algorithm is:

- Honest — sends its private input to the trusted party (i.e., sets $y_i = x_i$), and its only output is the value it gets from the trusted party (i.e., z_i).

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

An oracle-aided algorithm is:

- Honest — sends its private input to the trusted party (i.e., sets $y_i = x_i$), and its only output is the value it gets from the trusted party (i.e., z_i).
- Semi-honest, sends its input to the trusted party, outputs z_i plus possibly additional information. Might ask the trusted party to abort.

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

An oracle-aided algorithm is:

- Honest — sends its private input to the trusted party (i.e., sets $y_i = x_i$), and its only output is the value it gets from the trusted party (i.e., z_i).
- Semi-honest, sends its input to the trusted party, outputs z_i plus possibly additional information. Might ask the trusted party to abort.
- Malicious — acts arbitrarily.

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

An oracle-aided algorithm is:

- Honest — sends its private input to the trusted party (i.e., sets $y_i = x_i$), and its only output is the value it gets from the trusted party (i.e., z_i).
- Semi-honest, sends its input to the trusted party, outputs z_i plus possibly additional information. Might ask the trusted party to abort.
- Malicious — acts arbitrarily.

Ideal model execution

For oracle-aided algorithms $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, inputs $x_1, \ldots, x_n \in \{0,1\}^*$ and a function $f = (f_1, \ldots, f_n)$, let $\mathsf{IDEAL}^f_{\widehat{\pi}}(x_1, \ldots, x_n)$ denote the joint output of the parties in the end of the following experiment:

1. The input of $\widehat{\mathsf{P}}_i$ is x_i.
2. $\widehat{\mathsf{P}}_i$ sends some y_i to the trusted party .
3. Trusted party sends $z_i = f_i(y_1, \ldots, y_n)$ to $\widehat{\mathsf{P}}_i$ in an arbitrary order.
   - after receiving its output, a party can instruct trusted party to abort: not send the output to other party.
4. Each party outputs some value.

An oracle-aided algorithm is:

- Honest — sends its private input to the trusted party (i.e., sets $y_i = x_i$), and its only output is the value it gets from the trusted party (i.e., z_i).
- Semi-honest, sends its input to the trusted party, outputs z_i plus possibly additional information. Might ask the trusted party to abort.
- Malicious — acts arbitrarily.

Dishonest parties (semi-honest or malicious) can cooperate

Secure computation

Definition 1 (Secure computation)

A PPT protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ t-securely computes f, if $\forall$ PPT protocol $\pi = (\mathsf{P}'_1, \ldots, \mathsf{P}'_n)$ with at most t dishonest parties with respect to π, exists oracle-aided tuple $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, with at most t dishonest parties with respect to f, s.t.

$$\{\mathsf{REAL}_{\pi'}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n} \approx_c \{\mathsf{IDEAL}^f_{\widehat{\pi}}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n}$$

In case π' is honest, we require $\widehat{\pi}$ is honest, and ensembles to be identical.

Secure computation

Definition 1 (Secure computation)

A PPT protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ t-securely computes f, if $\forall$ PPT protocol $\pi = (\mathsf{P}'_1, \ldots, \mathsf{P}'_n)$ with at most t dishonest parties with respect to π, exists oracle-aided tuple $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, with at most t dishonest parties with respect to f, s.t.

$$\{\mathsf{REAL}_{\pi'}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n} \approx_c \{\mathsf{IDEAL}^f_{\widehat{\pi}}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n}$$

In case π' is honest, we require $\widehat{\pi}$ is honest, and ensembles to be identical.

For any efficiently commutable f, we present protocol for f that is $(t < n/2)$-secure against semi-honest adversities.

- Achieves perfect security: the distribution are identical, against unbounded adversaries

Secure computation

Definition 1 (Secure computation)

A PPT protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ t-securely computes f, if $\forall$ PPT protocol $\pi = (\mathsf{P}'_1, \ldots, \mathsf{P}'_n)$ with at most t dishonest parties with respect to π, exists oracle-aided tuple $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, with at most t dishonest parties with respect to f, s.t.

$$\{\mathsf{REAL}_{\pi'}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n} \approx_c \{\mathsf{IDEAL}^f_{\widehat{\pi}}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n}$$

In case π' is honest, we require $\widehat{\pi}$ is honest, and ensembles to be identical.

For any efficiently commutable f, we present protocol for f that is $(t < n/2)$-secure against semi-honest adversities.

- Achieves perfect security: the distribution are identical, against unbounded adversaries
- Can be compiled to be computationally secure against malicious adversaries using the compiler we seen last talk.

Secure computation

Definition 1 (Secure computation)

A PPT protocol $\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$ t-securely computes f, if $\forall$ PPT protocol $\pi = (\mathsf{P}'_1, \ldots, \mathsf{P}'_n)$ with at most t dishonest parties with respect to π, exists oracle-aided tuple $\widehat{\pi} = (\widehat{\mathsf{P}}_1, \ldots, \widehat{\mathsf{P}}_n)$, with at most t dishonest parties with respect to f, s.t.

$$\{\mathsf{REAL}_{\pi'}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n} \approx_c \{\mathsf{IDEAL}^f_{\widehat{\pi}}(\mathbf{x})\}_{\mathbf{x}\in(\{0,1\}^*)^n}$$

In case π' is honest, we require $\widehat{\pi}$ is honest, and ensembles to be identical.

For any efficiently commutable f, we present protocol for f that is $(t < n/2)$-secure against semi-honest adversities.

- Achieves perfect security: the distribution are identical, against unbounded adversaries
- Can be compiled to be computationally secure against malicious adversaries using the compiler we seen last talk.
- Or, using more sophisticated means, can be compiled to achieve perfect security against malicious adversaries.

Section 2

Secret Sharing

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography
- Secret sharing Vs. Error correcting codes.

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography
- Secret sharing Vs. Error correcting codes.
- In the following let n denote the number of parties

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography
- Secret sharing Vs. Error correcting codes.

- In the following let n denote the number of parties
- Let $P(n)$ be the family of all subset of $[n]$, not including the empty set.

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography
- Secret sharing Vs. Error correcting codes.

- In the following let n denote the number of parties
- Let $P(n)$ be the family of all subset of $[n]$, not including the empty set.
- Access structure is simply a set $\mathcal{A} \subseteq P(n)$

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography
- Secret sharing Vs. Error correcting codes.

- In the following let n denote the number of parties
- Let $P(n)$ be the family of all subset of $[n]$, not including the empty set.
- Access structure is simply a set $\mathcal{A} \subseteq P(n)$
- We will focus on monotone A: $\mathcal{I} \in \mathsf{A} \implies \mathcal{I}' \in \mathcal{A}$ for any $\mathcal{I}' \supseteq \mathcal{I}$

Secret sharing

A randomized mechanism that allows splitting shares a secret between a set of parties, such that only authorized set of parties can reconstruct the secret from their shares.

- A fundamental tool in cryptography
- Secret sharing Vs. Error correcting codes.

- In the following let n denote the number of parties
- Let $P(n)$ be the family of all subset of $[n]$, not including the empty set.
- Access structure is simply a set $\mathcal{A} \subseteq P(n)$
- We will focus on monotone A: $\mathcal{I} \in \mathsf{A} \implies \mathcal{I}' \in \mathcal{A}$ for any $\mathcal{I}' \supseteq \mathcal{I}$
- $\mathcal{A} = \{\mathcal{I} \subseteq [n] \colon |\mathcal{I}| \geq t\}$ is called the t-out-of-n access structure

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: For any $x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)_{\mathcal{I}}) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: For any $x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)_{\mathcal{I}}) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

- E is randomized

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: For any $x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)_{\mathcal{I}}) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

- E is randomized
- We focus on efficient (poly time) schemes for t-out-of-n access structure

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: For any $x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)_{\mathcal{I}}) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

- E is randomized
- We focus on efficient (poly time) schemes for t-out-of-n access structure
- Example: scheme for the n-out-of-n access structure over $\mathcal{D} = \{0, 1\}$:

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: For any $x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)_{\mathcal{I}}) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

- E is randomized
- We focus on efficient (poly time) schemes for t-out-of-n access structure
- Example: scheme for the n-out-of-n access structure over $\mathcal{D} = \{0, 1\}$:
  - $\mathsf{E}(x)$: outputs $z \leftarrow \{0, 1\}^n$ such that $\bigoplus_{i \in [n]} z_i = b$

Secret sharing schemes

Definition 2 (Secret sharing schemes)

A secret sharing scheme over domain $\mathcal{D}$ for access structure $\mathcal{A} \subseteq P(n)$, is a pair of algorithms (E, D) such that the following holds:

Correctness: For any $x \in \mathcal{D}$:

$$\mathsf{E}(x) \text{ outputs } z \in \Sigma^n, \text{ and } \mathsf{D}(\mathcal{I}, \mathsf{E}(x)_{\mathcal{I}}) = x \text{ for any } \mathcal{I} \in \mathcal{A}.$$

Secrecy: $\forall x, x' \in \mathcal{D}$ and $\mathcal{I} \notin \mathcal{A}$: $\mathsf{E}(x)_{\mathcal{I}} \equiv \mathsf{E}(x')_{\mathcal{I}}$

- E is randomized
- We focus on efficient (poly time) schemes for t-out-of-n access structure
- Example: scheme for the n-out-of-n access structure over $\mathcal{D} = \{0, 1\}$:
  - $\mathsf{E}(x)$: outputs $z \leftarrow \{0, 1\}^n$ such that $\bigoplus_{i \in [n]} z_i = b$
  - $\mathsf{D}([n], z)$: outputs $\bigoplus_i z_i$

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ by its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0, \ldots, \alpha_d)$.

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ by its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0, \ldots, \alpha_d)$.
- p is of degree (at most) d, if $\alpha_i = 0$ for all $i > d$.

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ by its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0, \ldots, \alpha_d)$.
- p is of degree (at most) d, if $\alpha_i = 0$ for all $i > d$.
- Sampling a random degree d polynomial p with $p(0) = x$, is done by setting $\alpha_0 = x$, and $(\alpha_1, \ldots, \alpha_d) \leftarrow \mathbb{F}^d$.

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ by its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0, \ldots, \alpha_d)$.
- p is of degree (at most) d, if $\alpha_i = 0$ for all $i > d$.
- Sampling a random degree d polynomial p with $p(0) = x$, is done by setting $\alpha_0 = x$, and $(\alpha_1, \ldots, \alpha_d) \leftarrow \mathbb{F}^d$.

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ by its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0, \ldots, \alpha_d)$.
- p is of degree (at most) d, if $\alpha_i = 0$ for all $i > d$.
- Sampling a random degree d polynomial p with $p(0) = x$, is done by setting $\alpha_0 = x$, and $(\alpha_1, \ldots, \alpha_d) \leftarrow \mathbb{F}^d$.
- Addition: Let p, p' be two degree $d < n$ polynomial over $\mathbb{F}$, and let $z, z' \in \mathbb{F}^n$ be their evaluations over $[n]$.

  Then $z + z'$, point-wise addition, is the evaluations values of the degree d polynomial q with $q(0) = p(0) + p'(0)$.

Polynomial over a finite field

Fix finite field $\mathbb{F}$ of size at least $n+1$, and identify $n+1$ values in $\mathbb{F}$ with $\{0, 1, \ldots, n\}$.

- We identify a polynomial $p \in \mathbb{F}[x]$ by its coefficient: i.e., $p(x) = \sum_{i=0}^{k} \alpha_i x^i$ is identifies by $(\alpha_0, \ldots, \alpha_d)$.
- p is of degree (at most) d, if $\alpha_i = 0$ for all $i > d$.
- Sampling a random degree d polynomial p with $p(0) = x$, is done by setting $\alpha_0 = x$, and $(\alpha_1, \ldots, \alpha_d) \leftarrow \mathbb{F}^d$.
- Addition: Let p, p' be two degree $d < n$ polynomial over $\mathbb{F}$, and let $z, z' \in \mathbb{F}^n$ be their evaluations over $[n]$.

  Then $z + z'$, point-wise addition, is the evaluations values of the degree d polynomial q with $q(0) = p(0) + p'(0)$.
- Multiplication: Let p, p' be two degree $d < n/2$ polynomial over $\mathbb{F}$, and let $z, z' \in \mathbb{F}^n$ be their evaluation over $[n]$.

  Then $z * z'$, point-wise multiplication, is the evaluations values of the degree $2d$ polynomial q with $q(0) = p(0) \cdot p'(0)$.

Shamir t-out-of-n scheme over $\mathbb{F}$

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t(z, \mathcal{I})$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t(z, \mathcal{I})$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:
  1. Find the polynomial p of degree $t-1$ such that $p(i) = z_i$ for all $i \in \mathcal{I}$

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t(z, \mathcal{I})$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:
  1. Find the polynomial p of degree $t-1$ such that $p(i) = z_i$ for all $i \in \mathcal{I}$
  2. Output $p(0)$.

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t(z, \mathcal{I})$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:
  1. Find the polynomial p of degree $t-1$ such that $p(i) = z_i$ for all $i \in \mathcal{I}$
  2. Output $p(0)$.

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t(z, \mathcal{I})$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:
  1. Find the polynomial p of degree $t-1$ such that $p(i) = z_i$ for all $i \in \mathcal{I}$
  2. Output $p(0)$.
- Correctness: for t-size $\mathcal{I} \subset [n]$, there exists a single degree $t-1$ polynomial with $p(i) = z_i$ for all $i \in \mathcal{I}$.

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0) = x$
  2. Output $(p(1), \ldots, p(n))$.
- $\mathsf{D}^t(z, \mathcal{I})$ for t-size $\mathcal{I} \subset [n]$, and $z \in \mathbb{F}^t$:
  1. Find the polynomial p of degree $t-1$ such that $p(i) = z_i$ for all $i \in \mathcal{I}$
  2. Output $p(0)$.
- Correctness: for t-size $\mathcal{I} \subset [n]$, there exists a single degree $t-1$ polynomial with $p(i) = z_i$ for all $i \in \mathcal{I}$.
- Efficiency: p can be found efficiently (e.g., *Lagrange interpolation*)

Shamir t-out-of-n scheme over $\mathbb{F}$

- $\mathsf{E}^t(x)$:
  1. Sample random polynomial p of degree $t-1$ such that $p(0)=x$
  2. Output $(p(1),\ldots,p(n))$.
- $\mathsf{D}^t(z,\mathcal{I})$ for t-size $\mathcal{I}\subset[n]$, and $z\in\mathbb{F}^t$:
  1. Find the polynomial p of degree $t-1$ such that $p(i)=z_i$ for all $i\in\mathcal{I}$
  2. Output $p(0)$.
- Correctness: for t-size $\mathcal{I}\subset[n]$, there exists a single degree $t-1$ polynomial with $p(i)=z_i$ for all $i\in\mathcal{I}$.
- Efficiency: p can be found efficiently (e.g., *Lagrange interpolation*)
- Secrecy: for any $\mathcal{I}\subset[n]$, of size $<t$, the value of $p(0)$ is uniform in $\mathbb{F}$, condition on $p(i)=z_i$ for all $i\in\mathcal{I}$

Section 3

BGW Multi-Party Protocol

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.
- Arithmetic circuit: gates, wires, inputs, outputs, values, computation

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.
- Arithmetic circuit: gates, wires, inputs, outputs, values, computation
- Fix a single output functionality $f : \mathbb{F}^n \mapsto \mathbb{F}$ computed by a poly-size arithmetic circuit C.

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.
- Arithmetic circuit: gates, wires, inputs, outputs, values, computation
- Fix a single output functionality $f \colon \mathbb{F}^n \mapsto \mathbb{F}$ computed by a poly-size arithmetic circuit C.

  We later discuss f computed by Boolean circuit

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.
- Arithmetic circuit: gates, wires, inputs, outputs, values, computation
- Fix a single output functionality $f \colon \mathbb{F}^n \mapsto \mathbb{F}$ computed by a poly-size arithmetic circuit C.

  We later discuss f computed by Boolean circuit

- We present perfectly secure protocol for computing f, against t semi-honest adversaries

The settings

- Fix $n \in \mathbb{N}$, $t < n/2$ and finite field $\mathbb{F}$ with $|\mathbb{F}| \geq n + 1$.
- Arithmetic circuit: gates, wires, inputs, outputs, values, computation
- Fix a single output functionality $f \colon \mathbb{F}^n \mapsto \mathbb{F}$ computed by a poly-size arithmetic circuit C.

  We later discuss f computed by Boolean circuit

- We present perfectly secure protocol for computing f, against t semi-honest adversaries
- Additions and multiplications are over $\mathbb{F}$

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   1.1 Input wires:

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i sets its share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i sets its share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.
   - **1.3** Multiplication gates: next slide

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i sets its share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.
   - **1.3** Multiplication gates: next slide
2. Each P_i send its share o_i of the (single) output wire to all parties.

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i sets its share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.
   - **1.3** Multiplication gates: next slide
2. Each P_i send its share o_i of the (single) output wire to all parties.
3. Each party output $\mathsf{D}^{t+1}([n], (o_1, \ldots, o_n))$

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i sets its share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.
   - **1.3** Multiplication gates: next slide
2. Each P_i send its share o_i of the (single) output wire to all parties.
3. Each party output $\mathsf{D}^{t+1}([n], (o_1, \ldots, o_n))$

The protocol

Protocol 3 ($\pi = (\mathsf{P}_1, \ldots, \mathsf{P}_n)$)

P_i**'s input:** $x_i \in \mathbb{F}$

1. The parties computes, in bottom-up order, shares for all wires of C:
   - **1.1** Input wires:
     - **1.1.1** P_i samples $z_i \leftarrow \mathsf{E}^{t+1}(x_i)$ and sends $(z_i)_j$ to P_j.
     - **1.1.2** P_i sets its share of the j'th input wire to $(z_j)_i$.
   - **1.2** Addition gates: P_i sets its share of the output wire to $a_i + b_i$, for a_i and b_i being its shares of the gate's input wires.
   - **1.3** Multiplication gates: next slide
2. Each P_i send its share o_i of the (single) output wire to all parties.
3. Each party output $\mathsf{D}^{t+1}([n], (o_1, \ldots, o_n))$

Claim 4 (Implies correctness of the protocol)

Let v be the value $\mathsf{C}(x_1, \ldots, x_n)$ assigns to a given wire, and let $Z_1, \ldots, Z_n$ be the value the above (random) protocol assign to the parties' shares of this wire. Then $\mathsf{D}^{t+1}([n], (z_1, \ldots, z_n)) = v$.

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow \mathsf{E}^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow \mathsf{E}^{g}(s_i)$. (let $s = \sum s_i$)

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow \mathsf{E}^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow \mathsf{E}^{g}(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow E^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow E^g(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow \mathsf{E}^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow \mathsf{E}^{g}(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$
5. P_i broadcasts $w_i = z_i + r_i^{2t+1}$.

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow E^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow E^g(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$
5. P_i broadcasts $w_i = z_i + r_i^{2t+1}$.
6. P_1 broadcasts $y = E^{t+1}((s + a \cdot b) = D^{2t+1}([n], (w_1, \ldots, w_n)))$

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow E^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow E^g(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$
5. P_i broadcasts $w_i = z_i + r_i^{2t+1}$.
6. P_1 broadcasts $y = E^{t+1}((s + a \cdot b) = D^{2t+1}([n], (w_1, \ldots, w_n)))$
7. P_i sets its share of the output wire to $o_i = y_i + r_i^{t+1}$

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow E^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow E^g(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$
5. P_i broadcasts $w_i = z_i + r_i^{2t+1}$.
6. P_1 broadcasts $y = E^{t+1}((s + a \cdot b) = D^{2t+1}([n], (w_1, \ldots, w_n)))$
7. P_i sets its share of the output wire to $o_i = y_i + r_i^{t+1}$

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow \mathsf{E}^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow \mathsf{E}^{g}(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$
5. P_i broadcasts $w_i = z_i + r_i^{2t+1}$.
6. P_1 broadcasts $y = \mathsf{E}^{t+1}((s + a \cdot b) = \mathsf{D}^{2t+1}([n], (w_1, \ldots, w_n)))$
7. P_i sets its share of the output wire to $o_i = y_i + r_i^{t+1}$

Claim 6

$\mathsf{D}^t([n], (o_1, \ldots, o_n)) = \mathsf{D}^t([n], (a_1, \ldots, a_n)) \cdot \mathsf{D}^t([n], (b_1, \ldots, b_n))$

Multiplication gates

Protocol 5 (Mult)

1. P_i sets $z_i = a_i \cdot b_i$, for a_i and b_i being its shares of the gate's input wires.
2. P_i samples $s_i \leftarrow \mathbb{F}$, sets $v_i^{t+1} \leftarrow \mathsf{E}^{t+1}(s_i)$ & $v_i^{2t+1} \leftarrow \mathsf{E}^{g}(s_i)$. (let $s = \sum s_i$)
3. P_i sends $((v_i^{t+1})_j, (v_i^{2t+1})_j)$ to P_j
4. P_i sets $r_i^{t+1} = \sum (v_j^{t+1})_i$ and $r_i^{2t+1} = \sum (v_j^{2t+1})_i$
5. P_i broadcasts $w_i = z_i + r_i^{2t+1}$.
6. P_1 broadcasts $y = \mathsf{E}^{t+1}((s + a \cdot b) = \mathsf{D}^{2t+1}([n], (w_1, \ldots, w_n)))$
7. P_i sets its share of the output wire to $o_i = y_i + r_i^{t+1}$

Claim 6

$\mathsf{D}^t([n], (o_1, \ldots, o_n)) = \mathsf{D}^t([n], (a_1, \ldots, a_n)) \cdot \mathsf{D}^t([n], (b_1, \ldots, b_n))$

Claim 7

Mult t-securely computes the functionality that on input $(a_1, b_1), \ldots, (a_n, b_n)$ sends $\mathsf{E}^t(\mathsf{D}^t([n], (a_1, \ldots, a_n)) \cdot \mathsf{D}^t([n], (b_1, \ldots, b_n)))_i$ to party P_i.

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $P_1, \ldots, P_t$ are corrupted.

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $z_{t+1}, \ldots, z_n$ uniformly at random conditioned on $\mathsf{D}^t([n], (z_1, \ldots, z_n)) = o$
4. Complete the execution of P_i, and output what it does.

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $z_{t+1}, \ldots, z_n$ uniformly at random conditioned on $\mathsf{D}^t([n], (z_1, \ldots, z_n)) = o$
4. Complete the execution of P_i, and output what it does.

Main observation: Wires' shares seen by corrupted parties have same distribution in real and ideal experiment.

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $z_{t+1}, \ldots, z_n$ uniformly at random conditioned on $\mathsf{D}^t([n], (z_1, \ldots, z_n)) = o$
4. Complete the execution of P_i, and output what it does.

Main observation: Wires' shares seen by corrupted parties have same distribution in real and ideal experiment.
Proof: Clear for *input wires*.

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $z_{t+1}, \ldots, z_n$ uniformly at random conditioned on $\mathsf{D}^t([n], (z_1, \ldots, z_n)) = o$
4. Complete the execution of P_i, and output what it does.

Main observation: Wires' shares seen by corrupted parties have same distribution in real and ideal experiment.
Proof: Clear for *input wires*. For *gates output wires*, proof by induction on the gate *depth*.

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $z_{t+1}, \ldots, z_n$ uniformly at random conditioned on $\mathsf{D}^t([n], (z_1, \ldots, z_n)) = o$
4. Complete the execution of P_i, and output what it does.

Main observation: Wires' shares seen by corrupted parties have same distribution in real and ideal experiment.

Proof: Clear for *input wires*. For *gates output wires*, proof by induction on the gate *depth*.

1. Addition gates: local computation

Claim 8

Protocol 3 perfectly computes f against t semi-honest adversaries

Proof: Assume $\mathsf{P}_1, \ldots, \mathsf{P}_t$ are corrupted.

Algorithm 9 ($\widehat{\mathsf{P}}_i$ on input x_i)

1. Send x_i to the trusted party, and let o be its answer.
2. Jointly with the other corrupted parties, emulate the first step of $(\mathsf{P}_1(x_1), \ldots, \mathsf{P}_t(x_t), \mathsf{P}_{t+1}(0), \ldots, \mathsf{P}_n(0))$.
3. Choose the value of $z_{t+1}, \ldots, z_n$ uniformly at random conditioned on $\mathsf{D}^t([n], (z_1, \ldots, z_n)) = o$
4. Complete the execution of P_i, and output what it does.

Main observation: Wires' shares seen by corrupted parties have same distribution in real and ideal experiment.

Proof: Clear for *input wires*. For *gates output wires*, proof by induction on the gate *depth*.

1. Addition gates: local computation
2. Multiplication gates: Mult securely compute multiplication functionality

Extensions

Extensions

- Have we used the fact that $t < n/2$?

Extensions

- Have we used the fact that $t < n/2$?
- $t = n - 1$?

Extensions

- Have we used the fact that $t < n/2$?
- $t = n - 1$?

Extensions

- Have we used the fact that $t < n/2$?
- $t = n - 1$? Use OT-based computational $(n-1)$-secure Mult protocol
- Boolean f?

Section 4

Course Summary

What we did

See diagram

What we did not cover

- Fully homomorphic encryption and obfuscation.

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto
- Non-generic constructions : number theory, lattices

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto
- Non-generic constructions : number theory, lattices
- Impossibility results

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto
- Non-generic constructions : number theory, lattices
- Impossibility results
- “Real life cryptography" (e.g., Random oracle model)

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto
- Non-generic constructions : number theory, lattices
- Impossibility results
- “Real life cryptography" (e.g., Random oracle model)
- Security

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto
- Non-generic constructions : number theory, lattices
- Impossibility results
- “Real life cryptography" (e.g., Random oracle model)
- Security
- Differential privacy

What we did not cover

- Fully homomorphic encryption and obfuscation.
- “Few" reductions
- Environment security (e.g., UC)
- Information theoretic crypto
- Non-generic constructions : number theory, lattices
- Impossibility results
- “Real life cryptography" (e.g., Random oracle model)
- Security
- Differential privacy
- and....

Advanced course (next semester, same time)

- Impossibility results
- Computation notion of entropy and their applications
- and more...

The exam

- 3 questions, each with several sub-questions
- Covers all material